Communication Protocol

API Development Documentation Protocol

smAiT Inc.

08/2025

Table of Contents

Protocol Overview

This protocol is primarily used for data interaction with the algorithm layer of the general chassis, enabling users to display data and issue commands. Network communication is implemented via WebSocket, with data exchanged in JSON string format.

WebSocket belongs to the server-side push technology, which is essentially an application layer protocol that can realize full-duplex two-way communication of persistent connections. The HTTP protocol is a one-way communication protocol, and the server will return data only if the client initiates an HTTP request. The WebSocket protocol is a two-way communication protocol, and after establishing a connection, both the client and the server can actively send or receive data to each other. The advantages are as follows:

1. Do not send HTTP requests frequently, only need to send an HTTP request for websocket handshake, and then you can use the TCP connection to communicate through the websocket protocol, avoiding the waste of transmitting multiple HTTP headers.
2. The full-duplex channel breaks through the limitation that only the client can request the server when requesting HTTP, and the server can directly push messages to the client.
3. It can be passed while generating data, and there is no need to wait for data generation to be completed, which improves efficiency.
4. WebSocket data transmission is based on data frames and can be transmitted in fragments, without fear that the data is too large and the packet cannot be accommodated.

Protocol Version Description

The version number follows the format major.minor.patch to distinguish protocol versions.

- Major indicates the protocol introduced when a new algorithm version is released.
- Minor represents significant updates within a major version, such as adding new fields or making substantial changes to field structures.
- Patch corresponds to minor updates within a major.minor version that do not affect the overall protocol structure, such as adjusting field content or refining descriptions.

Version 1.0.0
Infrastructure information and protocol addition of main data, some basic algorithm knowledge required for data analysis.

Common Communication Format

The computer communication program mainly has the following operations, which belongs to the general protocol format. All commands are formatted as follows, except that some fields
differ in content. Specific differences can be found in the sections of the Specific Data Agreement section.

- Publish information registration
- Publish specific data
- Log out of the publication information
- Subscribe to topic data
- Data return
- Unsubscribe from topic data
- Call the service interface
- Publish information registration

Publish Information Registration

Before the client publishes data to the server, it needs to register the publication information, the topic to be published, and the corresponding data type with the server. This registration only needs to be done once, typically during the initialization phase of the program.

```
{ "op": "advertise",
   (optional)    "id":    <string>   ,
"topic":        ,<string>
   "type":     <string>
}
```

Note:

"op": is a fixed value and must be "advertise".

"id": is optional and represents the currently registered ID number, used to distinguish between different client connections.

"topic": is required, but its value varies depending on the function. The corresponding values for each function are described in the Specific Data Protocol section.

"type": is required, but the value varies depending on the function. The corresponding values of each function are described in the Specific Data Protocol section.

Publish Specific Data

Publishing is valid only after the upper computer registers the publication information with the server. Without registration, the server will not respond to any publishing commands.

```
{ "op": "publish",
   (optional)   "id":    <string>  ,
"topic":       ,<string>
   "msg":   <json>
}
```

Note:

"op": is a fixed value and must be "publish".

"id": is optional and represents the currently registered ID number, used to distinguish between different client connections.

"topic": is required, but its value varies depending on the function. The corresponding values for each function are described in the Specific Data Protocol section.

"msg": is required, but the value varies depending on the function. The specific values of each function are described in the Specific Data Agreement section.

Unregistering Publication Messages

The upper computer needs to unregister the publication from the server when the topic data no longer needs to be published. This is typically done at the end of the program. However, since the connection is usually terminated at that stage, the server will automatically perform the deregistration. While this step is not mandatory, it is recommended to include it for the sake of logical consistency.

```
{ "op": "unadvertise", , "topic":
   (optional)   "id":    <string>  }
              <string>
```

Note:

"op": is a fixed value and must be "unadvertise".

"id": is optional and represents the currently registered ID number, used to distinguish between different client connections.

"topic": is required, but its value varies depending on the function. The corresponding values for each function are described in the Specific Data Protocol section.

Subscribe to Topic Data

The client needs to subscribe to information from the server, meaning that the server will publish certain information to the client.

```
{ "op": "subscribe",
   (optional)   "id":    <string> ,
"topic":       ,<string>
"type":       ,<string>
   (optional)   "throttle_rate":        <int>                    ,
   (optional)   "fragment_size":        <int>                    ,
   (optional)   "compression":      <string>
}
```

Note:

"op": is a fixed value and must be " subscribe".

"id": is optional and represents the currently registered ID number, used to distinguish between different client connections.

"topic": is required, but its value varies depending on the function. The corresponding values for each function are described in the Specific Data Protocol section.

"type": is required, but the value varies depending on the function. The corresponding values of each function are described in the Specific Data Protocol section.

"throttle_rate": is an optional value, specified in milliseconds (default is 0). It defines the return interval between subscription messages and can be used to adjust the frequency of the acquired data.

"fragment_size"**:** is an optional value that sets the split threshold for the returned JSON string. If the length of the returned string exceeds this value, the data will be unpacked. For details on the unpacked data format, see the corresponding subsection on data return.

"compression": is an optional value (default is no compression). When set to *png*, it enables compression for map and image transmission using the PNG compression method. Specific use cases requiring this function will be explicitly indicated; otherwise, this value should not be transmitted to the server

Data Return Frequency

After the client publishes the subscription information for a given topic to the server, the server returns the corresponding data to the client. The update frequency is not fixed: some data is returned at a fixed rate, some at variable rates, and some only occasionally. Although a *throttle_rate* value can be specified when publishing the subscription message, this value represents only the maximum frequency. If the original message frequency is lower, the data will still be published according to its original rate.

Complete Package Data Reception

If the amount of subscription data does not exceed the specified *fragment_size*, the data will be returned as a complete package.

```
{ "op": "publish",
"msg":      ,<string>
   "topic":      <string>
}
```

Note:

"op": is a fixed value and must be " publish".

"id": is optional and represents the currently registered ID number, used to distinguish between different client connections.

"topic": is required, but its value varies depending on the function. The corresponding values for each function are described in the Specific Data Protocol section.

Fragmented Data Reception

If the amount of subscribed data exceeds the specified fragment_size, the data will be split into multiple packets, each with a maximum size of fragment_size, and returned accordingly.

```
{ "op": "fragment",
"id":       ,<string>
"topic":         ,<string>
"data":       ,<string>
"num":      ,<int>
   "total":      <int>
}
```

Note:

"op": is a fixed value and must be "fragment". It indicates that the current packet is part of a split package.

"id": is the identification number, which will always be returned in this mode. It is used to distinguish split data from different clients and prevent data confusion.

"total": represents the total number of packets into which the data has been split. It specifies how many packets make up a complete data package.

"num": indicates the sequence number of the current packet within the split set, identifying its position in the sequence.

"data": contains part of the split data content. Do not parse this field individually! It is an incomplete string. After receiving all split packets, merge the contents of the *data* fields in order to reconstruct the full string. This reconstructed string is also JSON-structured. Once merged, parse it as JSON to obtain the complete data, which corresponds to the content described in *Complete Data Reception*. Then, apply the appropriate parsing method as required.

Unsubscribe from Topic Data

When the client no longer needs to receive messages for a given topic from the server, it sends an unsubscribe request to the server. After this, the server will stop sending data for that topic to the client.

```
{ "op": "unsubscribe", , "topic":
   (optional)    "id":    <string>  }
              <string>
```

Note:

"op": is a fixed value and must be "unsubscribe".

"id": is an optional value representing the currently registered ID number, used to distinguish between different client connections.

"topic": is required, and its value depends on the specific function. The corresponding values for each function are described in the *Specific Data Protocol* section.

Service Request Interface

This mode follows a request–response pattern. After the client publishes a service call request, the server returns the corresponding response data. However, the response time is not fixed.

```
{ "op": "call_service",
   (optional)    "id":    <string>  ,
"service":        ,<string>
   (optional)    "args":    <list<json>>                    ,
}
```

Note:

"op": is a fixed value and must be "call_service".

"id": is optional and represents the ID of the current service request. It is used to distinguish requests made at different times or from different clients.

"service": is required and specifies the name of the service being called. The valid values vary depending on the function and are defined in the *Specific Data Protocol* section.

"args": contains the parameters passed when invoking the service. The values vary depending on the function and are defined in the *Specific Data Protocol* section. This field must be in JSON format.

Service Response Data

This section describes the response data returned by the service request interface. The response time is not fixed and may vary. Responses can be distinguished using the *id* field.

```
{ "op": "service_response",
   (optional)   "id":    <string>  ,
"service":        ,<string>
   (optional)   "values":     <list<json>>
                <boolean>
                                                  ,
   "result":
}
```

Note:

"op": is a fixed value and must be "service_response".

"id": is optional and represents the ID of the corresponding service request. It is used to distinguish requests made at different times or from different clients.

"service": is required and specifies the name of the called service. The valid values vary depending on the function and are defined in the *Specific Data Protocol* section.

"values": contains the specific results returned by the request. The values vary depending on the function and are defined in the *Specific Data Protocol* section.

"result": indicates the execution result of the corresponding request. This is a boolean value: true means the service request was executed successfully, while false indicates an error occurred. In the case of an error, the corresponding error message will be returned in the *values* field.

Specific Data Protocol

Subscription Category (client gets data from server)

This section describes how the client obtains data from the server. The general protocol involved mainly refers to Subscribing to Topic Data, Data Return, and Unsubscribing from Topic Data.

The process follows this rule: the client first publishes the corresponding subscription topic data, after which the server sends the data to the client. If the client no longer wishes to receive the data, it must send an unsubscribe command for that topic.

The callback frequency of the subscribed data may vary depending on the server configuration. The possible modes are as follows:

1. **Timed Callback (Fixed Frequency)**: Data is returned at a constant rate.
2. **Callback on State Change**: Data is returned only when a state change occurs, and the timing is not fixed.
3. **Fixed When Valid**: Data is returned only when updates occur. The frequency is kept as stable as possible, but delays may still happen.

Obtain Robot Coordinates (Timing Trigger)

This instruction is used to obtain the robot's coordinate information on the map (x, y, theta).
The first step is to send a subscription command to broadcast the data.

{
"op": "subscribe",
"id": "get_pose",
"topic": "/robot_pose",
"type": "geometry_msgs/Pose2D"
}
	op: Operation Type
	advertise: broadcast
	id: Identification number + random number
	topic: Message name
	type: Message type

Return the following data from the server side(receive):

{
"topic": "/robot_pose",
"msg": {
"y": -0.047,
"x": 0.009,
"theta": 0.0114
},
"op": "publish"
}
	topic: Message name
	msg: Message
	x: Global coordinate X-axis position (in the map), in meters
	y: Global coordinate Y-axis position (in the map), in meters
	theta: Direction in global coordinates (in map), in radians

When unsubscribing the message, send the following instruction (send):

{
"op": "unsubscribe",
"id": "get_pose",
"topic": "/robot_pose"
}
	op: Operation type
	advertise: Broadcast message
	id: Identification number + random number
	topic: Message name
	type: Message type

Acquisition of Radar Point Cloud Data (timing trigger)

This instruction is used to obtain radar data values in the robot's coordinate system (x, y).
The first step is to send a subscription command to broadcast the data (send).

{
"topic": "/laser_data",
"type":
"yutong_assistance/point_array",
"id": "get_laser",
"op": "subscribe"
}
	op: Operation type
	advertise: Broadcast message
	id: Identification number + random number
	topic: Message name
	type: Message type

Return the following data from the server side.(receive)

{
"topic": "/laser_data",
"msg": {
"px": [-0.276, 0.399, 1.634, 6.780, 3.219, 0.940, -0.103, -1.092, -2.927, -6.910, -2.495, -0.965],
"py": [-1.197, -1.205, -1.213, -0.415, 1.718, 1.741, 1.747, 1.756, 1.770, 0.340, -1.178, -1.191],
"pt": [0, 0, 0]
},
"op": "publish"
}
	topic: Message name
	msg: Message
	px: The radar coordinates correspond to the X-axis position in meters
	py: The radar coordinates correspond to the Y-axis position, in meters
	Pt: Represents invalid data in the radar point cloud, given as an array with a default value of 0.
	
	The lengths of px and py should be the same, although not fixed. In this example program, many point cloud points were removed for easier operation.
	
	The values of the px and py arrays above are based on the robot's perspective. To convert them into the map representation, they must be combined with the robot's position. For details on the fusion process, refer to the formulas described in "Radar Point Cloud Conversion to the Global Coordinate System."

If the subscription to this message is no longer needed, send the following instruction (send):

{
"topic": "/laser_data",
"type": "yutong_assistance/point_array",
"id": "get_laser",
"op": "unsubscribe"
}
	op: Operation type
	advertise: Message
	id: Identification number + random number
	topic: Message name
	type: Message type

Obtain Navigation Planning Paths (Fixed When Valid)

This command is to get the representation of the planned path in the map (x,y)
First Instruction to Subscribe to Broadcast (Send):

{
"topic": "/global_path",
"type": "yutong_assistance/point_array",
"id": "get_path",
"op": "subscribe"
}
	op: Operation type
	advertise: Message
	id: Identification number + random number
	topic: Message name
	type: Message type

The following data is returned from the server(Receiving):

{
"topic": "/global_path",
"msg": {
"px": [-5.199, -5.202, -5.198, -5.1948771774, -5.190, -5.186, -5.182],
"py": [-3.299, -3.255, -3.230, -3.206, -3.181, -3.156, -3.132, -3.107, -3.083, -3.060, -3.036],
"pt": [0, 0, 0]
},
"op": "publish"
}
	op: Operation type
	msg: Message
	px: The path corresponds to the X-axis position in meters in the map coordinates
	py: The path corresponds to the Y axis position in meters in the map coordinates
	pt: reserved field, not in use.
	
	The length of px and py should be the same. It's not fixed, though. This example program removes a lot of point clouds for easy operation.

If you do not need to subscribe to the message, send the following instruction(Send):

{
"topic": "/global_path",
"type": "yutong_assistance/point_array",
"id": "get_laser",
"op": "unsubscribe"
}
	op: Operation type
	advertise: Message
	id: Identification number + random number
	topic: Message name
	type: Message type

Obtain Map (Fixed When Valid)

This command is used to obtain map information. The map is transmitted in the form of an image.

Send the following instruction (send):

{
"compression": "png",
"topic": "/map",
"fragment_size": 6000,
"type": "nav_msgs/OccupancyGrid",
"id": "get_map",
"op": "subscribe"
 	
}
	op: Operation type
	advertise: Message
	id: Identification number + random number
	topic: Message name
	type: Message type
	fragment_size: Unpack string length threshold, if the data is too large, it will be split and sent by subpacket, the recommended value is 6000
	compression: Compression use PNG

If you do not need to subscribe to the message, send the following instruction(Send):

{
"topic": "/map",
"type": "nav_msgs/OccupancyGrid",
"id": "get_map",
"op": "unsubscribe"
}
	op: Operation type
	advertise: Message
	id: Identification number + random number
	topic: Message name
	type: Message type

Received from the network port in the following two possible returned data formats:

Complete Package Reception (Receive)

<table>
<tr>
<td>
<pre>
{
    "topic": "/map",
    "msg": {
        "info": {
            "origin": {
                "position": {
                    "y": -9.6,
                    "x": -9.6
                },
                "orientation": {
                    "y": 0,
                    "x": 0,
                    "z": 0,
                    "w": 1
                }
            },
            "width": 384,
            "resolution": 0.05,
            "height": 384
        },
        "data":
"iVBORw0KGgoAAAANSUhEUgAAAYAAAAGAC
AAAAACBrOpjAAAKQ0lEQVR4Ae3BAY4l2W1E0
RvcUYL7X0Igd0R4pseSlZb6N8xXbeYUeI7MmiS
zJsmsSTJrksyaJLMmyaxJMmuSzJoksybJrEkya5L
MmiSzJsmsSTJrksyaJLMmyaxJMmuSzJoksybJrE
kya5LMmiSzJsmsSTJrksyaJLMmyaxJMmuSzJoks
ybJrEkya5LMmiSzJsmsSTJrksyaJLMmyaxJMmuS
zJoksybJrEkya5LMmiSzJsms\\/6s0f0rzlzRtMh3J
VzM\\/kU7+BkyPTEMWATidTqgAnBVABT84+Tk
nPzjBV1TgNP8mTTorTFYU3FcURHFfvEyYHpmG
LL7azX908fcQpkemIYuvdV83P3XdF28XpkemIYs
vdvNBFm8XpkemIYuvFeaDLN4uTI9MQxZf607z
c1m8XZgemYYsvlaYD7J4uzA9Mg1ZfK3A\\/Fw
WbxemR6Yhi68V5oMs3i5Mj0xDFI\\/rTvNzWb
xdmB6Zhiy+VmB+Lou3C9Mj05DF1wrzQRZvF6Z
HpiGLr3Wn+bks3i5Mj0xDFI8rMD+XxduF6ZFpy
OJr3XySxduF6ZFpyOJrhfkgi7cL0yPTkMXXCszPZ
fF2YXpkGrL4WmE+yOLtwvTINGTxtcJ8kMXbhe
mRacjia4X5IIu3C9Mj05DF1wrzQRZvF6ZHpiGLL
xZ8UrxdmB6ZhizWQ5gemYYs1kOYHpmGLNZD
mB6ZhizWQ5gemYYs1kOYHpmGLNZDmB6Zhiz
WQ5gemYYs1kOYHpmGLNZDmB6ZhizWQ5ge
mYYs1kOYHpmGLNZDmB6ZhizWQ5gemYYs1k
OYHpmGLNZDmB6ZhizWQ5gemYYs1kOYHpm
GLNZDmB6ZhizWQ5gemYYs1kOYHpmGLNZDm
B6ZhizWQ5gemYYs1kOYHpmGLNZDmB6ZhizW
Q5gemYYs1kOYHpmGLNZDmB6Zhiz+Xu40WTf
JTxRnwvTINGTxWsFPOPlXhjT\\/kMWRMD0yD
Vm8VpiGLI6E6ZFpyOK1wjRkcSRMj0xDFq8VpiG
</pre>
</td>
<td>
topic: Message name
op:png
msg: cartographic information
Info: Map related information

Origin/position: The global coordinate value represented by the first pixel value

Origin/orientation:
This can be ignored, default

Width: The width of the map
Height: The height of the map

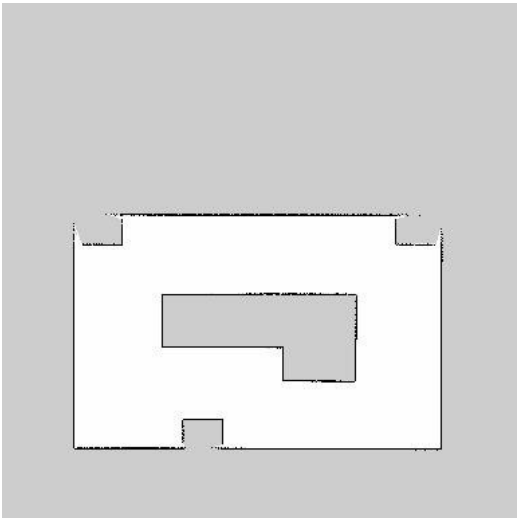
Resolution：Resolution of the map, one pixel represents
the actual size, example 0.05m one pixel lattice.

Data: data (base64 encoding), decoding should be directly using the image reading tool library can directly parse the data display.
</td>
</tr>
</table>

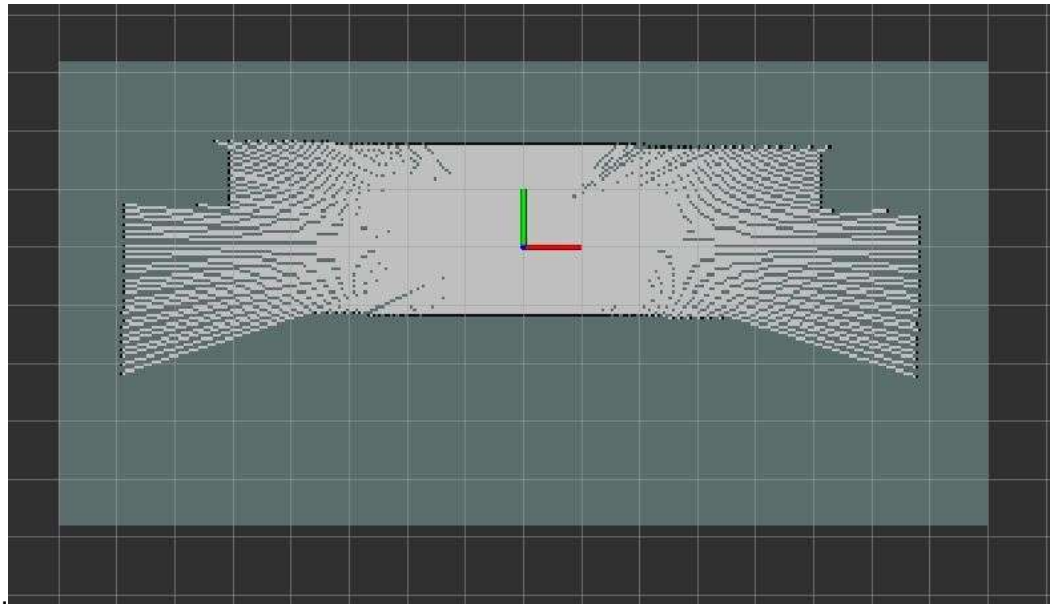
Ll2F6ZBqyeK0wDVkcCdMj05DFa4VpyOJImB6Z hixeK0xDFkfC9Mg0ZPFaYRqyOBKmR6Yhi9cK05 DFkTA9Mg1ZvFaYhiyOhOmRacjitcI0ZHEkTI9M QxavFaYhiyNhemQasnitMA1ZHAnTI9OQxWuF acjiSJgemYYsXitMQxZHwvTINGTxWmEasjgSpke mIYvXCtOQxZEwPTINWbxWmIYsjoTpkWnI4rXC NGRxJEyPTEMWrxWmIYsjYXpkGrJ4rTANWRwJ 0yPTkMVrhWnI4kiYHpmGLF4rTEMWR8L0yDR k8VphGrI4EqZHpiGLV7mv+wLurKgwDVkcCdMj 05DFoeC3MQ1ZHAnTI9OQxaEwr5LFkTA9Mg1Z HArzKlkcCdMj05DFoTCvksWRMD0yDVkcCvMq WRwJ0yPTkMWhMK+SxZEwPTINWRwK8ypJcS JMj0xDFofCvEsWJ8L0yDRkcSjMqyTFiTA9Mg1Z HArzKklxIkyPTEMWh8K8SxYnwvTINGRxKMyrJ MWJMD0yDVkcCvMqSXEiTI9MQxaHwrxLFifC9 Mg0ZHEozKskxYkwPTINWRwK8ypJcSJMj0xDFo fCvMoVxYkwPTINWRwK8ypJcSJMj0xDFofCvEs WJ8L0yDRkcSjMu2RxIkyPTEMWh8K8SIKcCNMj 05DFoTDvksWJMD0yDVkcCvMuWZwI0yPTkM WhMO+SxYkwPTINWRwK8ypJcSJMj0xDFofCvE sWJ8L0yDRkcSjMu2RxIkyPTEMWh8K8SxYnwvT INGRxKMy7ZHEiTI9MQxaHwrxKUpwI0yPTkM WhMO+SxYkwPTINWRwK8y5ZnAjTI9OQxaEw7 5LFiTA9Mg1ZHArzLlmcCNMj05DFoTCvkhQnwv TINGRxKMy7ZHEiTI9MQxaHwrxLFifC9Mg0ZHE ozLtkcSJMj0xDFofCvMoVxYkwPTINWRwKvo5J\ \/lB3gpMfnPyTkw+c\\/MkXJ8L0yDRk8f\\/hvvg 398V9ERUUYa6ggv\\/hK5xOfvAVdWdFBTiLO+u +7uu+4CYpuK8ovkKYHpmGLNZDmB6ZhizWQ5 gemYYs1kOYHpmGLNZDmB6ZhizWQ5gemYYs1 kOYHpmGLNZDmB6ZhizWQ5gemYYs1kOYHpm GLNZDmB6ZhizWQ5gemYYs1kOYHpmGLNZDm B6ZhizWQ5gemYYs1kOYHpmGLNZDmB6ZhizW Q5gemYYs1kOYHpmGLNZDmB6ZhizWQ5gemY Ys1kOYHpmGLNZDmB6ZhizWQ5gemYYs1kNg WmQasIgPgWmRachiPYTpkWnIYj0EpkWmIYv1 EKZHpiGL9RCYFpmGLNZDmB6ZhizWQ2BaZBq yWA9hemQasIgPYXpkGrJYD4FpkWnIYj2E6ZFpy OJPwQ9O\\/pPi+wgn\\/+TkXzgLCEyLTEMWfw qTTpNOp9P8Jc0fsvg2wllEEb6ibi64L+6LqPu6r\\ /uCMD0yDVn84eaTLL6NMJ9cEKZHpuPiDzefZP FthPnkgjtNi0xDFnDzURbfRpiPLsL0yDRkATcfZfF thPno4k7TItOQBdx8lMW3EeajizA9Mg1ZcPNZF t9GmI8uuOmRaci6r5vPsvg2wnx2BaZFpuO6r5v Psvg2wnx23TTJtFw3v5DFtxHmsytMj0xD+rr5hS y+jTC\\/cN30yDRk3fxKFt9GmF9I0yPTcd38Shbf RpjfRaYh6+ZXsvg2wvwuMg3XzS9l8W2E+V1kG tL82sW3EeZ3kflNsvg2wvwuMr9JFt9GmN9F5jf J4tsI87vl\\/BYX38zN7yHz9S6+p5uvJ\\/PFLoi6r 8DXfUE4676iCJP84AQq+Dtx+rohzV\\/SfA2ZL5 S+7nRWmOQ\\/c\\/K\\/OZ38kxOc\\/OCECv5S	

4QSc\\/IMTJ\\/\\/g5L85+YuTP5kEJzhxghOn02
nSaUjqTqcTTGLICvNDYrjCpEnz1WTWJJk1SWZN
klmTZNYkmTVJZk2SWZNk1iSZNUlmTZJZk2TWJ
Jk1SWZNklmTZNYkmTVJZk2SWZNk1iSZNUlmT
ZJZk2TWJJk1SWZNklmTZNYkmTVJZk2SWZNk1i
SZNUlmTZJZk2TWJJk1SWZNklmTZNYkmTVJZk2
SWZNk1iSZNUlmTZJZk2TWJJk1SWZNklmTZNYk
mTVJZk2SWZNk1iSZNUlmTZJZk2TWJJk1SWZNk
lmTZNYkmTVJZk2SWZNk1iSZNUlmTZJZk2TWJJ
k1SWZNklmTZNYkmTVJZk2SWZNk1iSZNUlmTZ
JZk2TWJJk1SWZNklmTZNYkmTVJZk2SWZNk1iS
ZNUlmTZJZk2TWJJk1SWZNklmTZNYkmTVJZk2S
WZNk1iSZNUlmTZJZk2TWJJk1SWZNklmTZNYk
mTVJZk2SWZNk1iSZNUlmTZJZk2TWJJk1SWZNk
lmTZNYkmTVJZk2SWZNk1iSZNUlmTZJZk2TWJJ
k1SWZNklmTZNYkmTVJZk2SWZNk1iSZNUlmTZ
JZk2TWJJk1SWZNklmTZNYkmTVJZk2SWZNk1iS
ZNUlmTZJZk2TWpP8Cm\\/MBPYlTy9QAAAAAS
UVORK5CYII="
    },
    "op": "png"
}

The data field contains the image data. The original map is first compressed using PNG, then encoded in Base64. Therefore, when parsing, the steps are reversed: first perform Base64 decoding on the data field, and then use the image library of the programming language (which should support directly reading PNG-compressed image data) to display the image.



The corresponding map display is shown above. (This image has been flipped, so the pixel coordinate (0,0) is located at the lower-left corner.) Based on the info field in the message, the world coordinate of the lower-left corner is (-9.6, -9.6). The image dimensions are 384 × 384, with each pixel grid representing 0.05 m.



In the figure above, the height is 160 and the width is 320. This is only an example to illustrate the map's dimensions. The world coordinate corresponding to the origin (lower-left corner) is (-8.0, -4.8). The large border grids represent a length of 1 m, and the red-green marker indicates the position of the world coordinate origin (0,0).

Fragmented Package Reception

The following data is returned from the server.

{
"total": 4,
"num": 0,
"data": "{xxxxx}",
"id": 0,
"topic": "/map",
"op": "fragment"
 	
}
	op: Identifies that the current packet is a part of the split package
	Id: Returns the ID set during subscription. It is used to distinguish unpacked information between different clients or different time periods.
	topic: Which topic data corresponds to the current unpacking data package, according to the host
	requirements
	total: How many packages in total make up a package
	Num: Identifying thecurrent packet is the number one in the split package
	Data: (Important)Do not parse the data field individually. It is an incomplete string. First receive all split packets, then concatenate the data fields in order to reconstruct the full string. The reconstructed string is JSON-structured; parse it as JSON to obtain the complete payload (equivalent to the content returned in Complete Package Reception). Subsequent processing follows the same steps as for the complete package.

Robot global state

First instruction subscribe to broadcast (send)

{
"topic": "/robot_status",
"type":
"yutong_assistance/RobotStatus",
"id": "get_robot_status",
"op": "subscribe"
}
	op: Operation type
	advertise: Message
	id: Identification number + random number
	topic: Message name
	type: Message type

Return the following data from the server side.(receive)

{
"topic": "/robot_status",
"msg": {
"current_building_name":
"tefa",
"soft_estop": false,
"battery": 0,
"charger": 0,
"nav_status": 0,
"patrol_status": 0,
"current_floor_name": "3",
"current_goal_coordinate": {
"y": 0.0,
"x": 0.0,
"theta": 0.0
},
"hard_estop": false,
"velocity": [0.0, 0.0],
"control_state": 30,
"current_goal_name": ""
},
"op": "publish"
}
	topic: The message name.
	msg: The message content.
	current_building_name(str): The building name corresponding to the current map.
	current_floor_name(str): The floor name corresponding to the current map.
	soft_estop(bool): Soft emergency stop; true indicates the robot is in an emergency stop state.
	hard_estop(bool): Hardware emergency stop; true indicates the robot is in an emergency stop state.
	battery(int): The percentage of the battery charge.
	charger (int): Charging status:
	• 0 – Not connected
	• 1 – Successfully charging
	• 2 – Recharge in progress
	• -1 – Recharge failed
	(see Recharge Result section for details)
	nav_status (int): Current navigation status:
	• 600 – Waiting
	• 601 – Running
	• 602 – Cancelled
	• 603 – Success
	• 604 – Failed
	(Refer to the Status Code Table for details)
	patrol_status (int):
	Multi-point navigation state.
	control_state (int): Current program state:
	• 20 – Mapping
	• 30 – Positioning / Navigation
	• 99 – Error
	velocity (list): An array where:
	• 1st element: Linear velocity (m/s)
	• 2nd element: Angular velocity (rad/s)
	current_goal_name(str): The name of the current navigation target point.

	If empty, it indicates the target was published using coordinate values. Corresponds to the marked point information. **current_goal_coordinate(dict)**: Specific coordinates of the current navigation point: **x**: X-axis value in the map's coordinate system (meters) **y**: Y-axis value in the map's coordinate system (meters) **theta**: Angle expression in the map (radians)

If not required to subscribe to this message, send the following instructions: (send)

{
"topic": "/robot_status",
"type":
"yutong_assistance/RobotStatus",
"id": "get_robot_status",
"op": "unsubscribe"
}
	op: Operation type
	advertise: Message
	id: Identification number + random number
	topic: Message name
	type: Message type

Navigation State(on state transition)

The instruction is to obtain the information involving navigation, is it running or stopped, what is the reason of the stop.

First instruction subscribe to broadcast (send)

{
"topic": "/navi_status",
"type": "actionlib_msgs/GoalStatus",
"id": "get_map",
"op": "subscribe"
}
	op: Operation type
	advertise: Message
	id: Identification number + random number
	topic: Message name
	type: Message type

Return the following data from the server side.(receive)

{
"topic": "/navi_status", "msg":
{
"status": 3,
"text": "Current goal
is success",
"goal_id": {
"stamp": {
"secs": 5976,
"nsecs": 489000000
},
"id":
"/move_base-4-5976.489000000"
}
},
"op": "publish"
}
	topic: Message name
	msg: Message
	status (int8): State information for the navigation
	• 0: Power-on initialization wait
	• 1: Target point in running
	• 2: The target point is cancelled by the user
	• 3: The target point was reached successfully
	• 4: Target point navigation failed
	
	Text (str): Specific message text
	Goal_id: The id number of the corresponding issued target point (currently negligible)

If not required to subscribe to this message, send the following instructions: (send)

{
"topic": "/navi_status",
"type": "actionlib_msgs/GoalStatus",
"id": "get_map",
"op": "unsubscribe"
}
	op: Operation type
	advertise: Message
	id: Identification number + random number
	topic: Message name
	type: Message type

Laser Obstacle Detection Area

The laser detects the area information of obstacles within a specified surrounding range. The command is to get where the obstacle is detected around the robot and force it to stop moving.

First instruction subscribe to broadcast (send)

{
"topic": "/obstacle_region",
"type": "std_msgs/Int8",
"id": "get_obstacle_region",
"op": "subscribe"
}
	op: Operation type
	advertise: Message
	id: Identification number + random number
	topic: Message name
	type: Message type

Return the following data from the server side.(receive)

{
"topic": "/obstacle_region",
"msg": {
"data": 2
},
"op": "publish"
}
	topic: Message name
	msg: Message
	data (int): Obstacle area information
	0: No obstacles around
	• 1: Obstacles within 90 degrees on the right side
	• 2: Obstacles within 90 degrees directly ahead
	• 4: Obstacles within 90 degrees on the left side

If not required to subscribe to this message, send the following instructions: (send)

{
"topic": "/obstacle_region",
"type": "std_msgs/Int8",
"id": "get_obstacle_region",
"op": "unsubscribe"
}
	op: Operation type
	advertise: Message
	id: Identification number + random number
	topic: Message name
	type: Message type

Map-Matching Confidence Scores

The map calculated by the current algorithm matches the confidence degree calculated by the lidar data. Values range from 0 to 1.0. Corresponding percentage form.

First instruction subscribe to broadcast (send)

{
"topic": "/localization_confidence",
"type": "std_msgs/Float64",
"id": "get_localization_confidence",
"op": "subscribe"
}
	op: Operation type
	advertise: Message
	id: Identification number + random number
	topic: Message name
	type: Message type

Return the following data from the server side.(receive)

{
"topic": "/localization_confidence",
"msg": {
"data": 0.6
},
"op": "publish"
}
	topic: Message name
	msg: Message
	data (double): Confidence score of the current lidar-to-map matching. The value ranges from 0 to 1.0 (corresponding to a percentage).

If not required to subscribe to this message, send the following instructions: (send)

{
"topic": "/localization_confidence",
"type": "std_msgs/Float64",
"id": "get_localization_confidence",
"op": "unsubscribe"
}
	op: Operation type
	advertise: Message
	id: Identification number + random number
	topic: Message name
	type: Message type

Map-Matching Security Protection Threshold

Returns the threshold where the current map match triggers security protection.
First instruction subscribe to broadcast (send)

{
"topic":
"/laser_safety_controller/confidence_threshol d",
"type": "std_msgs/Float64",
"id": "get_match_threshold",
"op": "subscribe"
}
	op: Operation type
	advertise: Message
	id: Identification number + random number
	topic: Message name
	type: Message type

Return the following data from the server side.(receive)

{
"topic": "/laser_safety_controller/confidence_threshol d",
"msg": {
"data": 0.6
},
"op": "publish"
}
	topic: Message name
	msg: Message
	data (double): Confidence score of the current lidar-to-map matching. Value range is 0 to 1.0 (corresponding to percentage form).

If not required to subscribe to this message, send the following instructions: (send)

{
"topic": "/laser_safety_controller/confidence_threshol d",
"type": "std_msgs/Float64",
"id": "get_match_threshold",
"op": "unsubscribe"
}
	op: Operation type
	advertise: Message
	id: Identification number + random number
	topic: Message name
	type: Message type

Active Push Notification

Actively push some system notifications. Specific notice content has some below

- Map matching threshold trigger and lidar data invalid trigger warning
- Chassis sensor is abnormal

First instruction subscribe to broadcast (send)

{
"topic": "/notification",
"type": "rosgraph_msgs/Log",
"id": "get_/notification",
"op": "subscribe"
}
	op: Operation type
	advertise: Message
	id: Identification number + random number
	topic: Message name
	type: Message type

Return the following data from the server side.(receive)

{
"topic": "/notification",
"msg": {
"function": "激光雷达",
"name": "system_monitor",
"level": 16,
"topics": ["laser"],
"header": {
"stamp": {
"secs": 1627005599,
"nsecs": 23596048
},
"frame_id": "",
"seq": 2
},
	topic: Message name
	msg: Message
	
	Fields:
	Name (str): The main category of the active prompt. Indicates which functional module initiated the push.
	Level (int): Event level:
	• 0: Prompt event
	• 1: Debug level
	• 2: Standard event
	• 4: Warning event
	• 8: Error event
	• 16: Fatal event

"file": "", "msg": "激光雷达未连接", "line": 0 }, "op": "publish" }	**Topics (array)**: Event list. Currently, the chassis sensor returns the names of individual sensors. **Header:** **Stamp**: Timestamp information (seconds since 1970-01-01). • Secs (int): Seconds • Nsecs (int): Nanoseconds **Msg**: Detailed message content. For the detailed definition of active notification information, refer to the appendix "Active Notification Content and Field Details."

If not required to subscribe to this message, send the following instructions: (send)

{
"topic": "/notification",
"type": "rosgraph_msgs/Log",
"id": "get_/notification",
"op": "unsubscribe"
}
	op: Operation type
	advertise: Message
	id: Identification number + random number
	topic: Message name
	type: Message type

Chassis Sensor Data

The chassis is equipped with sensors that provide status information such as:

- Battery power (percentage of remaining charge)
- Voltage
- Hard emergency stop
- Charge and discharge state
- Charging pile connection state

The **charge and discharge state** reflects the actual internal battery charging or discharging condition. For example, when the robot is on the charging pile, it may show as "not charging" if the internal overcharge protection has been triggered, even though it is docked.

The **charging pile connection state**, on the other hand, simply indicates whether the robot is physically connected to the charging pile, regardless of whether the battery is actively charging or discharging.

First instruction subscribe to broadcast (send)

{
"topic": "/mobile_base/sensors/core",
"type": "kobuki_msgs/SensorState",
"id": "get_sensors_core",
"op": "subscribe"
}
	op: Operation type
	advertise: Message
	id: Identification number + random number
	topic: Message name
	type: Message type

Return the following data from the server side.(receive)

<table>
<tr>
<td>

```
{
   "topic": "/mobile_base/sensors/core",
   "msg": {
         "current": "AAA=",
         "right_encoder": 1503,
         "analog_input": [0, 39319, 0, 0],
         "bottom": [0, 0, 0],
         "battery": 97,
         "charger": 1,
    }
         "over_current": 1,
         "left_pwm": -29,
         "buttons": 0,
         "header": {
               "stamp": {
                     "secs": 1627033096,
                      "nsecs": 700239558
               },
               "frame_id": "",
               "seq": 279096
         },
         "right_pwm": 25,
         "wheel_drop": 0,
         "bumper": 0,
         "time_stamp": 27582,
         "digital_input": 0,
         "left_encoder": 63529,
         "cliff": 0
   },
   "op": "publish"
}
```

</td>
<td>

Topic: Message name
Msg: Message
Header:

- **Stamp** – Time-stamp information (seconds since 1970-01-01)
  - **Secs (int):** Seconds
  - **Nsecs (int):** Nanoseconds

Fields:

- **Battery (int):** Battery charge percentage.
- **Charger (int):** Charging pile connection status (whether the robot is on the charging pile).
  - 0 = not on charging pile
  - 1 = on charging pile
- **Over_current (int):** Charge / discharge state.
  - 0 = not charging
  - 1 = charging
- **Buttons (int):** Hardware emergency stop state.
  - 0 = emergency stop not triggered
  - 1 = emergency stop triggered
- **Analog_input (array):** Array of ultrasound data in millimeters. Currently, only the second array value is valid, representing the central ultrasonic sensor.

</td>
</tr>
</table>

If not required to subscribe to this message, send the following instructions: (send)

<table>
<tr>
<td>

```
{
      "topic": "/mobile_base/sensors/core",
      "type": "kobuki_msgs/SensorState",
      "id": "get_sensors_core",
      "op": "unsubscribe"
}
```

</td>
<td>

op: Operation type
advertise: Message
id: Identification number + random number
topic: Message name
type: Message type

</td>
</tr>
</table>

Release class (Client → Server)

General Protocol Formats:

- Registration of release information
- Publishing specific data
- Cancellation of release information

Notes:

- Registration of release information only needs to be published **once during initialization**.
- After registration, the client can continuously publish specific data content.

Coordinate System:

- All coordinate-related data published below are expressed in the **map coordinate system**.
- If the command contains a **header** field, its **frame_id** subfield must be set to **map**.

Speed Control Command

- The robot is controlled by issuing the corresponding control speed, which is a differential drive mode (linear speed, angular velocity).
- A single release speed lasts for 0.6 seconds. Finally stop with a full zero speed.
- Do not always publish zero speed, the algorithm layer will think that the user continues to use manual remote control mode, resulting in no navigation response.

Broadcast first (send)

{
"op": "advertise",
"id": "velocity_control",
"topic": "/cmd_vel_mux/input/teleop",
"type": "geometry_msgs/Twist"
}
	op: Operation type
	advertise: Message
	id: Identification number + random number
	topic: Message name
	type: Message type

Send data to the "/cmd_vel_mux/input/teleop" message. Circular Send (Send)

{
"op": "publish",
"topic": "/cmd_vel_mux/input/teleop",
"id": "velocity_control",
"msg": {
"angular": {
"z": 0
},
"linear": {
"x": 0.2
}
}
}
	msg (object): Message
	
	angular (float): Angular velocity
	Unit: radians per second (rad/s).
	
	z (float): Rotation speed
	Example: z: 0 → no rotation.
	
	linear (float): Linear velocity
	Unit: meters per second (m/s).
	
	x (float): Forward speed
	Example: x: 0.2 → forward motion at 0.2 m/s.

Stop publishing the subscribe (send)

{
"op": "unadvertise",
"id": "velocity_control",
"topic": "/cmd_vel_mux/input/teleop"
}
	op: Indicates the message type
	advertise: Stop Broadcast
	id: Identification number + random number
	topic: Message name

The extended speed control command is roughly as follows:

- msg->linear->x is set to a positive value for linear advance. The unit is m/s; msg->angular->z is 0.
- msg->linear->x is negative when backing up.The unit is m/s; msg->angular->z is 0.
- For clockwise rotation in place, msg->linear->x is set to 0;msg->angular->z is a positive value in rad/s
- For counterclockwise rotation, msg->linear->x is set to 0;msg->angular->z is a negative value in rad/s
- To stop, set msg->linear->x to 0;msg->angular->z is 0

Cancel the Target Point Navigation

The main function of this instruction is to cancel the target point navigation.

Broadcast first (send)

{
"op": "advertise",
"id": "cancel_goal",
"topic": "/move_base/cancel",
"type": "actionlib_msgs/GoalID"
}
	op: Indicates the message type
	advertise: Broadcast
	id: Identification number + random number
	topic: Message name

Send data to the “/move_base/cancel” message. (send)

{
"op": "publish",
"topic": "/move_base/cancel",
"id": "cancel_goal",
"msg": {
"stamp": "",
"id": ""
}
}
	op: publish
	topic: Message name
	id: Identification number + random number
	msg: Message
	stamp: Time stamp (empty is fine)
	id: Target point id (empty is fine)

Stop publishing the subscribe (send)

{
"op": "unadvertise",
"id": "cancel_goal",
"topic": "/move_base/cancel"
}
	op: Indicates the message type
	advertise: Stop broadcasting
	id: Identification number + random number
	topic: Message name

Soft and Emergency stop

The main function of this instruction is to start the emergency stop function

Broadcast first (send)

{
"topic": "/soft_stop",
"type": "std_msgs/Bool",
"id": "set_estop",
"op": "advertise"
}
	op: Indicates the message type
	advertise: broadcasting
	id: Identification number + random number
	topic: Message name

Send data to the “/soft_stop” message. (send)

{
"topic": "/soft_stop",
"msg": {
"data": true
},
"id": "set_estop",
"op": "publish"
}
	msg: Message
	
	Data (bool): Set to true to enable the software emergency stop function, false to disable it

Stop publishing the subscribe (send)

{
"topic": "/soft_stop",
"id": "set_estop",
"op": "unadvertise"
}
	op: Indicates the message type
	advertise: Stop broadcasting
	id: Identification number + random number
	topic: Message name

Add Current Robot Position as a Marked Point

The main function of this instruction is to add the current location of the robot to the list of marked points.

Broadcast first (send)

{
"topic": "/insert_current_pose_marker",
"type": "yutong_assistance/InsertPose",
"id": "insert_current_pose_marker",
"op": "advertise"
}
	op: Indicates the message type
	advertise: broadcasting
	id: Identification number + random number
	topic: Message name

Send data to the /insert_current_pose_marker message. (Send)

{ "topic": "/insert_current_pose_marker", "msg": { "time_out": 0, "behavior_code": 11, "name": "csyt2", "rest_time": 11 }, "id": "insert_current_pose_marker", "op": "publish" }	**msg**: Message **Behavior_code** (int): The property of the current marked point. Currently, 11 represents a recharge point. **Time_out** (float): The execution timeout for the current point, in seconds. If the point is not reached within the specified time, it will be canceled and the next point will be executed. A value of -1 means no timeout is set, and the system will wait until the point is completed. **Name** (str): The name corresponding to the current point. This must be unique; if not, one of the duplicates will be overwritten. **Rest_time** (float): The rest time at the current point, in seconds. After reaching this point, the system waits for the specified time before continuing to the next point.

Stop publishing the subscribe (send)

{ "topic": "/insert_current_pose_marker", "id": "insert_current_pose_marker", "op": "unadvertise" }	**op**: Indicates the message type **advertise**: stop broadcasting **id**: Identification number + random number **topic**: Message name

Return to the map immediately

The main function of this instruction is used to trigger the map data callback immediately

Broadcast first (send)

{ "topic": "/get_current_map ", "type": "std_msgs/Bool", "id": "get_current_map", "op": "advertise" }	**op**: Indicates the message type **advertise**: broadcasting **id**: Identification number + random number **topic**: Message name

Send data to the “/get_current_map” message. (Send)

{ "topic": "/get_current_map", "msg": { "data": true }, "id": "get_current_map", "op": "publish" }	**msg**: Message **Data** (bool): Set to true to trigger the immediate callback of map data.

Stop publishing the subscribe (send)

{
"topic": "/get_current_map",
"id": "get_current_map",
"op": "unadvertise"
}
	op: Indicates the message type
	advertise: stop broadcasting
	id: Identification number + random number
	topic: Message name

Laser Detection Stops the Movement

The main function of this instruction is to stop the manual control when the laser detects an obstacle

Broadcast first (send)

{
"topic": "/node_manager/laser_safety_controller",
"type": "std_msgs/Bool",
"id": "set_laser_detection",
"op": "advertise"
}
	op: Indicates the message type
	advertise: broadcasting
	id: Identification number + random number
	topic: Message name

Send data to the /node_manager/laser_safety_controller message. (Send)

{
"topic": "/node_manager/laser_safety_controller",
"msg": {
"data": true
},
"id": "set_laser_detection",
"op": "publish"
}
	msg: Message
	data (bool):
	• **true**: Enable the stop movement function when surrounding obstacles are detected.
	• **false**: Disable the stop movement function when surrounding obstacles are detected.

Stop publishing the subscribe (send)

{
"topic": "/node_manager/laser_safety_controller",
"id": "set_laser_detection",
"op": "unadvertise"
}
	op: Indicates the message type
	advertise: stop broadcasting
	id: Identification number + random number
	topic: Message name

Laser Detection Distance

- The main function of this instruction is to set the distance value of the laser detection obstacle.
- The lowest distance value was 0.3 m. Below 0.3 m is set to the default lowest value of 0.3m.

Broadcast first (send)

{
"topic": "/node_manager/laser_safety_range",
"type": "std_msgs/Float64",
"id": "set_laser_safety_range",
"op": "advertise"
}
	op: Indicates the message type
	advertise: broadcasting
	id: Identification number + random number
	topic: Message name

Send data to the /node_manager/laser_safety_range message. (Send)

{
"topic": "/node_manager/laser_safety_range",
"msg": {
"data": 0.5
},
"id": "set_laser_safety_range",
"op": "publish"
}
	msg: Message
	
	data (double): Detection threshold for laser-based stop movement, in meters. If an obstacle is detected within this distance, the robot will stop moving.

Stop publishing the subscribe (send)

{
"topic": "/node_manager/laser_safety_range",
"id": "set_laser_safety_range",
"op": "unadvertise"
}
	op: Indicates the message type
	advertise: stop broadcasting
	id: Identification number + random number
	topic: Message name

Map Matching Security Protection Threshold Adjustment

- The main function of this instruction is to use the map-matching threshold to trigger safety protection.
- During navigation, if the map-matching confidence falls below this threshold:
  - A safety protection behavior will be triggered.
  - The chassis will rotate in place.
  - The robot will then stop moving.
  - Manual remote control will be invalid during this state.
- The restriction will only be released after the navigation ends.
- Default value: 0.8 (80%).

Broadcast first (send)

{
"topic": "/laser_safety_controller/setting_confidence_threshold",
"type": "std_msgs/Float64",
"id": "set_safety_match_threshold",
"op": "advertise"
}
	op: Indicates the message type
	advertise: broadcasting
	id: Identification number + random number
	topic: Message name

Send data to the /laser_safety_controller/confidence_threshold message. (Send)

{
"topic": "/laser_safety_controller/setting_confidence_threshold",
"msg": {
"data": 0.5
},
"id": "set_safety_match_threshold",
"op": "publish"
}
	msg: Message
	data (double): Detection threshold for triggering safety protection based on map matching.
	• When the confidence of map matching falls below this threshold, safety protection is triggered.
	• Threshold range: 0 ~ 1.0, corresponding to a percentage.

Stop publishing the subscribe (send)

{
"topic": "/laser_safety_controller/setting_confidence_threshold",
"id": "set_safety_match_threshold",
"op": "unadvertise"
}
	op: Indicates the message type
	advertise: stop broadcasting
	id: Identification number + random number
	topic: Message name

Service Class (Client Publishes Data to Server and Returns Corresponding Result)

This section is the request reply mode. The client sends the request information to the server, and the server returns the reply information to the client after performing the corresponding operation. The relevant common protocol sections are "Call Request Service Interface" and "Service Return Data".

Program Control Framework

The main function of this instruction is to build the map, locate the navigation, save the map, and switch the map.

The request corresponding to publishing the service (publishing)

{
"args": {
"floor_num": "2",
"cmd": 0,
"args": 0,
"building_name": "csst"
},
"id": "service_node_manager_control",
"service": "/node_manager_control",
"op": "call_service"
}
	op (str): "call_service" indicates that the operation is a request service
	id (str): Used to set the ID number corresponding to the service. It can be used to determine the return value corresponding to the response signal
	Service (str): The corresponding service name of the request
	Args (json): Input parameters corresponding to the service request

Parsed JSON data:
Building_name (str): The building name of the current map
Floor_num (str): The floor name of the current map
Cmd (int): The current corresponding action
- 0: Open mapping (available in positioning/navigation mode)
- 1: Continue mapping (reserved)
- 2: Restart mapping (reserved)
- 3: Save map and corresponding information (available in any mode)
- 4: Turn on location navigation (available in mapping mode)

7: Switch map (available in positioning/navigation mode)

Response returned by this service (receive)

<table>
<tr>
<td>{
"id":"service_node_manager_control",
"values": {
"text": "bringup mapping success!",
"result": 0
},
"result": true,
"service": "/node_manager_control",
"op": "service response"
}</td>
<td>op (str): "service_response" indicates that the operation is a reply to the request service
id (str): Used to verify the results of the corresponding requested service
service (str): The corresponding service name of the request
result (bool): Whether the currently requested service is performed correctly. If false, the value field returns the specific reason for the failure
value: Return result corresponding to the service request, defined as follows:
result (int): The result code for the current instruction execution. Error code definitions are listed in the table below
text (str): Additional information for the current instruction, used only for debug or log output</td>
</tr>
</table>

The error codes correspond to the table listed below

Error code	Corresponding information
0	Successful operation
99	Unknown operation
101	The program cannot be closed because it has not been started at all
102	Already in the current mode
107	Enter the missing map information
106	System fatal error (may require restart)
108	Map damage
109	Current operation is not supported (switching is not supported in mapping state)
110	Failed to save map

List the System Floor Maps and Summary Information

The main function of this instruction is to manage the maps and related virtual walls saved in the current system, and to mark the point information.
The request corresponding to publishing the service (publishing)

{ "args": {
"op": 0
},
"id": "service_layered_map_cmd",
"service": "/layered_map_cmd",
"op": "call_service"
 	
}
	op (str): "call_service" indicates that the operation is a requested service
	id (str): Used to set the corresponding ID number for the service. Can be used to determine the return value corresponding to the response signal
	service (str): The corresponding service name of the request
	args (json): Input parameters corresponding to the service request

Response returned by this service (receive)

{
"id": "service_layered_map_cmd",
"values": {
"info": "list map info success",
"list_info": [{
"floor_info": [{
"map": true,
"floor_name": "floor_1",
"markers": false,
"virtual_walls": false
}],
"building_name": "origin"
}, {
"floor_info": [{
"map": true,
"floor_name": "floor_1",
"markers": false,
"virtual_walls": false
}],
"building_name":
"\u6211\u7684\u7956\u56fd"
}, {
"floor_info": [{
"map": true,
"floor_name": "floor_4",
"markers": true,
"virtual_walls": true
}, {
	op (str): "service_response" indicates that the operation is a reply to the request service
	id (str): used to verify the results of the corresponding requested service
	service (str): the corresponding service name for the request
	result (bool): whether the currently requested service is performing correctly. if false, value will return the specific reason for the failure
	value: return answer result corresponding to the service
	info (str): the information attached to the current instruction, only for debug or log output
	result (int): the currently returned result code, see the error number in the table below
	ist_info (str): all maps and their associated information in the current system. only returned when the requested op is set to 0. details are as follows:
	building_name (str): building name
	floor_info (list): collection of all floor information under the current building
	floor_name (str): floor name

"map": true, "floor_name": "floor_1", "markers": false, "virtual_walls": false }], "building_name": "cs" }], "result": 0 }, "result": true, "service": "/layered_map_cmd", "op": "service_response" }	**map (bool):** identifies whether there is a map **virtual_walls** (bool): identifies whether there is currently virtual wall information **markers (bool):** identifies whether there is currently mark point information Note: When list is adopted, markers and virtual_walls are empty sets. Another form of return including markers and virtual_walls is provided below.

Parsed data format:

- Building_name (str): The building name of the current map
- Floor_name (str): The floor name of the current map
- Op (int): The current corresponding action
    - 0: Lists all the map information in the current system

Delete Map Information

The main function of this instruction is to delete a map of the currently specified floor or all maps under the entire building.

The request corresponding to publishing the service (publishing)

{
"args": {
"floor_name": "123",
"op": 0,
"building_name": "\u53bb\u53bb\u53bb"
},
"id": "service_building_operation/delete",
"service": "/building_operation/delete",
"op": "call_service"
}
	op (str): call_service indicates that the operation is a requested service
	id (str): used to set the id number corresponding to the service. can be used to determine the return value corresponding to the response signal
	service (str): the corresponding service name of the request
	args (json): input parameters corresponding to the service request

The data parsed from json are as follows:

- building_name (str): the building name of the current map
- floor_name (str): the floor name of the current map
- op (int): the current corresponding action
    - 0: delete the map of the given floor_name in the specified building_name
    - if floor_name is an empty string, delete all maps under the current building

Response returned by this service (receive)

<table>
<tr>
<td>
<pre>
{ "id": "service_building_operation/delete",
"values": {
"info": "this message success",
"list_info": [{
"floor_info": [{
"map": false,
"floor_name": "",
"markers": false,
"virtual_walls": false
}],
"building_name": ""
}],
"result": 0,
"markers": {
"waypoints": [{
"behavior_code": 0,
"time_out": 0.0,
"pose": {
"position": {
"y": 0.0,
"x": 0.0,
"z": 0.0
},
"orientation": {
"y": 0.0,
"x": 0.0,
"z": 0.0,
"w": 0.0
}
},
"name": "",
"rest_time": 0.0
}]
},
"vwalls": {
"map_name": "",
"header": {
"stamp": {
"secs": 361,
"nsecs": 312000000
},
"frame_id": "map",
"seq":
},
"walls": [{
"end_x": 0.0,
"wall_id": "",
"end_y": 0.0,
"start_x": 0.0,
"start_y": 0.0
}]
}
},
</pre>
</td>
<td>
op (str): service_response indicates that the operation is a reply to the request service

id (str): can be used to verify the results of the corresponding requested service

service (str): the corresponding service name for the request

result (bool): whether the currently requested service is performed correctly. if false, value returns the specific reason for the failure

value: return answer result corresponding to the service request, defined as follows

info (str): information attached to the current instruction, only for debug or log output

result (int): the result code returned for the current instruction. see the error code table below for definitions
</td>
</tr>
</table>

"result": true, "service": "/building_operation/delete", "op": "service_response" }	

Error Code	Meaning
0	Successful operation
201	Invalid path(parameter map_folder_base correspondence, default home/maps
202	No corresponding file / path under no valid map
203	No data / file read errors are present in the file
204	No permission; delete the file is not in the home directory
205	File is corrupted; delete path failed
206	Get map data timeout
207	Not known to read map (possibly under state)
208	Failed to save map
99	Unknown instructions

Get the List of Current Marker Points

The main function of this instruction is to load the list dot information corresponding to the current map.

The request corresponding to publishing the service (publishing)

{
"args": {
"op": 0
},
"id": "service_get_markers",
"service":
"/marker_operation/get_markers",
"op": "call_service"
}
	op (str): call_service indicates that the operation is a requested service
	id (str): used to set the corresponding id number for the service. can be used to determine the return value corresponding to the response signal
	service (str): the corresponding service name for the request
	args(json): the input parameters corresponding to the service request

The data resolved by json are as follows:

- building_name (str): the building name of the current map
- floor_name (str): the floor name of the current map
- op (int): the current corresponding action
    - 0: load the marker information corresponding to the current map

Response returned by this service (receive)

{
"id":
"service_marker_operation/get_markers",
"values": {
"info": "",
"list_info": [{
"floor_info": [{
	op (str): service_response indicates that the operation is a reply to the request service

```
                    "map": false,
                    "floor_name": "",
                    "markers": false,
                    "virtual_walls": false
                }],
                "building_name": ""
            }],
            "result": 0,
            "markers": {
                "waypoints": [{
                    "behavior_code": 0,
                    "time_out": 0.0,
                    "pose": {
                        "position": {
                            "y": 0.0,
                            "x": 0.0,
                            "z": 0.0
                        },
                        "orientation": {
                            "y": 0.0,
                            "x": 0.0,
                            "z": 0.0,
                            "w": 1.0
                        }
                    },
                    "name": "a1",
                    "rest_time":
                0.0 }, {
                    "behavior_code": 0,
                    "time_out": 0.0,
                    "pose": {
                        "position": {
                            "y": 3.0,
                            "x": 1.0,
                            "z": 0.0
},
                        "orientation": {
                            "y": 0.0,
                            "x": 0.0,
                            "z": 0.0,
                            "w": 1.0
                        }
                    },
                    "name": "a3",
                    "rest_time": 0.0
                }]
            },
            "vwalls": {
                "map_name": "",
                "header": {
                    "stamp": {
                        "secs": 5543,
                        "nsecs": 187000000
                    },
```


id (str): used to verify the results of the corresponding requested service

service (str): the corresponding service name for the request

result (bool): whether the currently requested service is performed correctly. if false, value returns the specific reason for the failure
value: return answer result corresponding to the service request, defined as follows

- **info** (str): information attached to the current instruction, used only for debug or log output
- **result** (int): the result code currently returned, the corresponding error number is shown in the error code table

Compatible mode message structure: only the following part needs to be parsed

markers.waypoints (list): collection of marked points. format is the same as in marker point information

- behavior_code (int): property of the current marker point. (11 = recharge point)
- time_out (float): execution timeout for the current point (seconds). if the point is not reached within this time, cancel it and move to the next point. set to -1 for no timeout (wait until complete)

pose: position and angle information of the current point

- position.x: the corresponding x-coordinate value in the map
- position.y: the corresponding y-coordinate value in the map

<table>
<tr>
<td>
<pre>
                "frame_id": "map",
                "seq": 0
            },
            "walls": [{
                "end_x": 0.0,
                "wall_id": "",
                "end_y": 0.0,
                "start_x": 0.0,
                "start_y": 0.0
            }]
        }
    },
    "result": true, "service":
"/marker_operation/get_markers",
    "op": "service_response"
}
</pre>
</td>
<td>

orientation: the quaternion value corresponding to the point angle in the map

name (str): the corresponding name of the current point, which must be unique. if not unique, one of them will be overwritten

rest_time (float): rest time at the current point. number of seconds to wait after reaching the point before proceeding to the next point
</td>
</tr>
</table>

Error code	Meaning
0	Successful operation
201	Invalid path (parameter map_folder_base correspondence, default home/maps)
202	No corresponding file / path under no valid map
203	No data / file read errors are present in the file
204	No permission; delete the file is not in the home directory
205	File is corrupted; delete path failed
206	Get map data timeout
207	Not known to read map (possibly under state)
208	Failed to save map
99	Unknown instructions

Remove a Single Marker Point (poi name pattern)

The main function of this instruction is to remove the mark point by setting the corresponding target point name in the mark point

The request corresponding to publishing the service(publishing)

<table>
<tr>
<td>
<pre>
{
    "args": {
        "poi": "p2"
    },
    "id":
"service_marker_manager/delete_poi",
    "service":
"/marker_manager/delete_poi",
    "op": "call_service"
}
</pre>
</td>
<td>
op (str): call_service indicates that the operation is a requested service
id (str): used to set the id number corresponding to the service. can be used to determine the return value corresponding to the response signal
service (str): the corresponding service name of the request
args (json): input parameters corresponding to the service request
The data resolved by json are as follows:
poi(str): Delete the name corresponding to the target point
</td>
</tr>
</table>

Response returned by this service (receive)

{
"id": "service_marker_manager/delete_poi",
"values": {
"avaliable_list": ["a1", "a3", "a2"],
"success": true
},
"result": true,
"service": "/marker_manager/delete_poi",
"op": "service_response"
}
	op (str): service_response marks that the operation is a requested service response
	
	id (str): can be used to verify the results of the corresponding requested service
	
	service (str): the corresponding service name of the request

result (bool): whether the currently requested service is performed correctly. if false, value returns the specific reason for the failure

- example: {"id":"service_XXX","values":"Service /poi does not exist","result":false,"service":"/poi","op":"service_response"}
- this means the service failed because the corresponding service on the algorithm program side was not started

value: the return response result corresponding to the service request, defined as follows:

- **success** (bool): whether the current target point was successfully published. if false, it means the corresponding point name does not exist in the current point list. refer to available_list for valid points
- **available**_list (list): a string-format list array containing the marker points set in the current corresponding map

Publish Navigation Point Information (poi name mode)

The main function of this instruction is to perform a single point navigation by setting the corresponding target point name in the tag point

The request corresponding to publishing the service(publishing)

{
"args": {
"poi": "P1"
},
"id": "service_poi",
"service": "/poi",
"op": "call_service"
}
	op (str): call_service marks that the operation is a requested service
	id (str): used to set the id number corresponding to the service. can be used to determine the return value corresponding to the response signal
	service (str): the corresponding service name of the request
	args (json): input parameters corresponding to the service request
	• the parsed data are as follows:
	○ **poi (str):** the name corresponding to the publication target point

Response returned by this service (receive)

{
"P3", "P0",
"id": "service_poi",
"values": {
"avaliable_list": ["P2",
"P1"],
"success": true
},
"result": true,
"service": "/poi",
"op": "service_response"
}
	op (str): service_response indicates that the operation is a reply to the requested service
	
	id (str): can be used to verify the results of the corresponding requested service
	
	service (str): the corresponding service name of the request

result (bool): whether the currently requested service is performed correctly. return the specific reason for the failure if false.value

- example: {"id":"service_XXX","values":"Service /poi does not exist","result":false,"service":"/poi","op":"service_response"}
- this means the service is not started on the algorithm program side

value: return result corresponding to the service request, defined as follows

- **success (bool):** whether the current target point was successfully published. if false, it means the corresponding point name does not exist in the current point list. refer to available_list to check currently available points
- **available_list (list):** a list array in string format containing the marker points set

Get Current Running Map Information Interface

A list array in string format that contains the marker points set in the current corresponding map.

The main function of this instruction is to obtain information about the current map.

The request corresponding to publishing the service(publishing)

{
"args": {
"cmd": 0
},
"id": "service_get_map_info",
"service": "/get_map_info",
"op": "call_service"
 	
}
	op (str): call_service indicates that the operation is a requested service
	id (str): used to set the id number corresponding to the service. Can be used to determine the return value corresponding to the response signal
	service (str): the corresponding service name of the request
	args (json): input parameters corresponding to the service request

The parsed data are presented as follows:

- **cmd (int):**
  - 0: getting data

Response returned by this service (receive)

{
"id": "service_get_map_info",
"values": {
"height": 256,
"width": 352,
"origin_y":-9.6000003815, "origin_x": -9.6000003815,
"free_space": 200.0,
"occ_space": 15.5,
"unknown_space": 10.28,
"floor_name": "1",
"resolution": 0.0500000007,
"building_name": "tefa"
 	
},
"result": true,
"service": "/get_map_info",
"op": "service_response"
}
	op (str): service_response indicates that the operation is a reply to the requested service
	
	id (str): can be used to verify the results of the corresponding requested service
	
	service (str): the corresponding service name of the request
	
	result (bool): whether the currently requested service is performed correctly. If false, value will return the specific reason for the failure

value: the service request corresponds to the returned response result. The specific content is as follows:

- **origin_x (double):** the global x-coordinate represented by the first pixel value
- **origin_y (double):** the global y-coordinate represented by the first pixel value
- **width (int):** the width of the map
- **height (int):** the height of the map
- **resolution (double):** the resolution of the map, where one pixel represents the actual size (e.g., 0.05 m per pixel grid)
- **building_name (str):** the building name corresponding to the current map
- **floor_name (str):** the floor name corresponding to the current map
- **free_space (double):** the blank area of the current map (m²)
- **occ_space (double):** the occupied area of the current map (m²)
- **unknown_space (double):** the unknown area of the current map (m²)
- **exploration area:** calculated as blank area + occupied area

Navigation Speed Modification

The main function of this instruction is to set the speed mode that needs to execute during navigation

The request corresponding to publishing the service(publishing)

{
"args": {
"cmd": 3,
"str": ""
},
"id": "service_velocity_control",
"service": "/velocity_control",
"op": "call_service"
}
	op (str): call_service marks the operation as a requested service
	id (str): used to set the id number corresponding to the service. can be used to determine the return value corresponding to the response signal
	service (str): the corresponding service name of the request

args (json): input parameters corresponding to the service request. the parsed data are as follows:

cmd (int): speed mode, divided into three gears (low, medium, high). each gear has three modes (safety, balance, efficiency). in total, there are nine possible combinations:

- **0:** safety low speed
- **1:** safety medium speed
- **2:** safety high speed
- **3:** balance low speed
- **4:** balance medium speed
- **5:** balance high speed
- **6:** efficiency low speed
- **7:** efficiency medium speed
- **8:** efficiency high speed
- **-1:** default mode (factory default parameter). this parameter is only for transitional reading. after setting, it takes effect on the next startup. this mode can also be considered balance medium speed
- **60:** smooth mode (food delivery mode), enable smooth mode
- **61:** turn off smooth mode (food delivery mode). when turned off, the speed is reset to default mode (-1)
- **99:** get the current speed mode

str (str): [reserved for additional parameters if applicable]

Response returned by this service (receive)

{
"info": "this message success",
"service": "/velocity_control", "values": {
"info": "balance mode with low speed",
"result": 0
},
"result": true,
"id": "service_velocity_control",
"op": "service_response"
}
	op (str): service_response marks the operation as a reply to the requested service
	id (str): can be used to verify the results of the corresponding requested service
	service (str): the corresponding service name of the request
	result (bool): whether the currently requested service is performed correctly. if false, value will return the specific reason for the failure

- **value:** the return answer result corresponding to the service request, defined as follows:
  - **info (str):** additional information of the current instruction, only for debug or log output. when the input is to get the current speed mode (cmd=99), the return string is numeric and represents the value of the current system speed mode
  - **result (int):** return result code
    - **0:** successful execution
    - **99:** unknown instruction
    - **620:** corresponding speed profile not found
    - **621:** dynamic modification of speed failed (specific reason given in info field)

Hardware Test

Chassis Sensor Self-Inspection

- **Main function:** Performs self-inspection of the chassis sensors to check whether the configured sensors are operating normally.
- **Process:** After the service is called, the system requires a short waiting period while it polls each sensor to verify proper functionality.
- **Sensors checked:**
  - Left and right encoders
  - Battery voltage
  - Charger connection
  - Ultrasonic sensors (left / middle / right)
  - LiDAR
  - Depth camera
  - Network connection
  - Gyroscope
  - Infrared recharge signal

The request corresponding to publishing the service (publishing)

{
"args": {},
"id": "service_self_diagnosis",
"service": "/self_diagnosis",
"op": "call_service"
}
	op (str): call_service indicates that the operation is a requested service
	id (str): used to set the id number corresponding to the service. can be used to determine the return value corresponding to the response signal

service (str): the corresponding service name of the request
args (json): no additional input parameters are available

Response returned by this service (receive)


```
{
    "info": "this message success",
    "service": "\/self_diagnosis", "values": {
        "status": [{
            "message": "激光雷达正常工作",
            "hardware_id": "激光雷达",
            "values": [],
            "name": "",
            "level": 0 }, {
            "message": "网络连接正常",
            "hardware_id": "网络连接",
            "values": [],
            "name": "",
            "level": 0 }, {
            "message": "编码器左传感器数据未收到",
            "hardware_id": "编码器左",
            "values": [],
            "name": "",
            "level": 2 }, {
            "message": "编码器右传感器数据未收到",
            "hardware_id": "编码器右",
            "values": [],
            "name": "",
            "level": 2 }, {
```

```
                "message": "电池电压传感器数据未收到",
                "hardware_id": "电池电压",
                "values": [],
                "name": "",
                "level": 2 }, {
                "message": "充电器连接传感器数据未收到",
                "hardware_id": "充电器连接",
                "values": [],
                "name": "",
                "level": 2 }, {
                "message": "红外信号数据未收到",
                "hardware_id": "红外回充信号",
                "values": [],
                "name": "",
                "level": 2 }, {
{
    "info": "this message success",
    "service": "\/self_diagnosis", "values": {
        "status": [{
                "message": "激光雷达正常工作",
                "hardware_id": "激光雷达",
                "values": [],
                "name": "",
                "level": 0 }, {
                "message": "网络连接正常",
                "hardware_id": "网络连接",
                "values": [],
                "name": "",
                "level": 0 }, {
                "message": "编码器左传感器数据未收到",
                "hardware_id": "编码器左",
                "values": [],
                "name": "",
                "level": 2 }, {
                "message": "编码器右传感器数据未收到",
                "hardware_id": "编码器右",
                "values": [],
                "name": "",
                "level": 2 }, {
                "message": "电池电压传感器数据未收到",
                "hardware_id": "电池电压",
                "values": [],
                "name": "",
                "level": 2 }, {
                "message": "充电器连接传感器数据未收到",
                "hardware_id": "充电器连接",
                "values": [],
                "name": "",
                "level": 2 }, {
                "message": "红外信号数据未收到",
                "hardware_id": "红外回充信号",
                "values": [],
                "name": "",
                "level": 2 }, {
```

```
                "message": "超声波左传感器数据未收到",
                "hardware_id": "超声波左",
                "values": [],
                "name": "",
                "level": 2 }, {
                "message": "超声波中传感器数据未收到",
                "hardware_id": "超声波中",
                "values": [],
                "name": "",
                "level": 2 }, {
                "message": "超声波右传感器数据未收到",
                "hardware_id": "超声波右",
                "values": [],
                "name": "",
                "level": 2 }, {
                "message": "陀螺仪数据正常",
                "hardware_id": "陀螺仪",
                "values": [],
                "name": "",
                "level": 0 }, {
                "message": "深度相机正常",
                "hardware_id": "深度相机",
                "values": [],
                "name": "",
                "level": 0
            }],
            "id": "",
            "passed": 0
        },
        "result": true,
        "id": "service_self_diagnosis",
        "op": "service_response"
                }
```


op (str): "service_response" — indicates that the operation is a reply to the request service.
id (str): Used to verify the results of the corresponding requested service.
service (str): The corresponding service name of the request.
result (bool): Indicates whether the requested service executed correctly.

- If false, the value field returns the specific reason for failure.

value: The return result corresponding to the service request. Includes the following details:

- **passed (byte):** 0 — no need to pay attention.
- **id (string):** "" — no need to pay attention.
- **status (list):** Array containing the states of multiple sensors. Each element includes:
    - **hardware_id (str):** Name of the sensor device.
    - **level (int):** Current status level of the device:
        - 0: normal operation
        - 1: warning
        - 2: error
        - 3: data timeout
    - **message (str):** Additional output information.
    - **name (str):** Specific error code.

Robot Version Information

The main function of this instruction is to obtain information about the hardware version of the robot software.

The request corresponding to publishing the service(publishing)

{ "args": { "cmd": 0 }, "id": "service_robot_info", "service": "/robot_info", "op": "call_service" }	**op (str):** "call_service" — marks the operation as a requested service. **id (str):** Used to set the corresponding ID number for the service. Can be used to determine the return value corresponding to the response signal. **service (str):** The corresponding service name of the request.

args (json): Input parameters corresponding to the service request.

- **cmd (int):** Version-related instruction.
- Definition: 0: obtain relevant information about the current robot chassis.

Response returned by this service (receive)

{ "info": "this message success", "service": "/robot_info", "values": { "robot_id": "TY1251C003_0036", "firmware_version": "1.0.2", "robot_type": "TONG", "software_version": "4.1.0-alpha", "hardware_version": "2.0.0" }, "result": true, "id": "service_robot_info", "op": "service_response" }	**op** (str): "service_response" — marks the operation as a reply to the requested service. **id** (str): Can be used to verify the results of the corresponding requested service. **service** (str): The corresponding service name of the request. **result** (bool): Indicates whether the requested service was performed correctly. • If false, the value field will return the specific reason for the failure.

value: The return data corresponding to the service request, defined as follows:

- **robot_id (str):** Robot number.
- **robot_type (str):** Robot type.
- **software_version (str):** Algorithm version.
- **firmware_version (str):** Firmware version.
- **hardware_version (str):** Hardware version.

Get Current Map Matching Threshold

The main function of this instruction is to obtain the threshold for map matching set in the current system.

The request corresponding to publishing the service(publishing)

{
"args": {
"cmd": 0,
"str": ""
},
"id": "service_get_match_threshold",
"service": "/get_match_threshold",
"op": "call_service"
 	
}
	op (str): "call_service" — marks the operation as a requested service.
	
	id (str): Used to set the corresponding ID number for the service. Can be used to determine the return value corresponding to the response signal.
	service (str): The corresponding service name of the request.

args (json): Input parameters corresponding to the service request.

- cmd (int): Version-related instructions.
  - 0: Get the map matching threshold set by the current system.

Response returned by this service (receive)

{
"info": "this message success",
"service": "/get_match_threshold",
"values": {
"info": "setting match threshold to 80 %",
"result": 80
},
"result": true,
"id": "service_get_match_threshold",
"op": "service_response"
}
	op (str): "service_response" — indicates the operation is a reply to the requested service.
	id (str): Can be used to verify the results of the corresponding requested service.
	service (str): The corresponding service name of the request.
	result (bool): Whether the requested service executed successfully.
	• If false, the value field returns the specific reason for the failure.
	value: The returned result of the service request.
	• **result (int):** The threshold value, given directly as a percentage.

Calculate the Distance Between Points

The main function of this instruction is to calculate the distance between the two points in the current map. This distance is the navigation distance planned by the path calculated from the map information.

The request corresponding to publishing the service(publishing)

{
"args": {
"goal_x": 1.1,
"goal_y": 1.2,
	op (str): "call_service" — marks the operation as a requested service.
	id (str): Used to set the ID number corresponding to the service. Can be used

"start_x": 0.1, "start_y": 0.2, "start_floor": "1", "goal_floor": "1" }, "id": "service_calculate_distance", "service": "/calculate_distance", "op": "call_service" }	to determine the return value of the response signal. **service (str):** The corresponding service name of the request. **args (json):** Input parameters for the service request. The resolved JSON data is as follows: • **start_x (double):** Starting point x-axis coordinate. • **start_y (double):** Starting point y-axis coordinate. • **start_floor (str):** Floor where the starting point is located. • **goal_x (double):** End point x-axis coordinate. • **goal_y (double):** End point y-axis coordinate. • **goal_floor (str):** Floor where the end point is located.

Response returned by this service (receive)

{
"info": "this message success",
"service": "/calculate_distance",
"values": {
"info": "",
"distance": 1.4309386487,
"result": 0
},
"result": true,
"id":
"service_calculate_distance",
"op": "service_response"
}
	op (str): "service_response" — marks the operation as a reply to the requested service.
	id (str): Used to verify the results of the corresponding requested service.
	service (str): The corresponding service name of the request.
	result (bool): Indicates whether the requested service was executed correctly.
	• If false, the value field provides the specific reason for failure.

value (object): The return data for the service request, defined as follows:

- **info (str):** Additional information for debugging or log output.
- **result (int):** Result code of execution:
  - 0 → Successful execution
  - 10 → Previous instruction still running
  - 150 → Path planning failed, destination unreachable
  - 151 → Path calculation server error
  - 152 → Cross-floor path calculation not supported
- **distance (double):** The path distance (in meters).
- **info (str):** Extra information provided when execution is unsuccessful, used for debugging.

Status code table

General Standard Output Status

- **0**: Instruction executed successfully
- **10**: Instruction is being processed
- **20**: Mapping in progress
- **30**: Localization and navigation in progress
- **99**: Unknown input instruction

Program Control Framework

- **101**: Error closing program, because it was never started
- **102**: Invalid startup request, program is already running
- **104**: No corresponding map available
- **105**: No permission to control the program
- **106**: System fatal error, please restart
- **107**: Missing building or floor information
- **108**: Map file corrupted
- **109**: Instruction not supported in current mode
- **110**: Failed to save map

Floor / Map / Virtual Wall / Marker / Area Management

(Floor management uses codes in the 2XX range, map management in 3XX, virtual wall in 4XX, markers in 50X, and area management in 55X. They share similar structures but differ in the hundred's digit.)

- **201**: Invalid path
- **202**: File not found
- **203**: No data content available
- **204**: No permission to access
- **205**: File corrupted
- **206**: Map data acquisition timeout
- **207**: Loading map not supported in mapping mode
- **208**: Failed to save data file

Navigation

- **600**: Initialization pending
- **601**: Running
- **602**: Navigation cancelled
- **603**: Navigation successful
- **604**: Navigation failed
- **605**: Navigation refused
- **606**: Cancelling navigation
- **607**: Navigation reset in progress
- **608**: Navigation reset completed
- **609**: Navigation status lost
- **620**: Configuration failed
- **621**: Emergency stop button triggered

Cross-Floor Navigation (requires elevator control)

- **630**: Invalid cross-floor navigation input
- **631**: Invalid target floor for cross-floor navigation
- **632**: Invalid target marker for cross-floor navigation
- **633**: Error obtaining cross-floor navigation marker point
- **634**: Invalid elevator-related points for cross-floor navigation
- **635**: Cross-floor navigation failed
- **636**: Cross-floor navigation cancelled
- **637**: Failed to reserve elevator for cross-floor navigation
- **639**: Waiting for cross-floor navigation
- **640**: Checking feasibility of cross-floor navigation
- **641**: Navigating to outside elevator
- **642**: Waiting for elevator to reach designated floor
- **643**: Entering elevator
- **644**: Switching map
- **645**: Leaving elevator
- **646**: Navigating to target point
- **647**: Cross-floor navigation completed

Multi-Point Navigation

- **650**: Multi-point navigation completed
- **651**: Multi-point navigation in progress
- **652**: Multi-point navigation paused
- **653**: Multi-point navigation resumed
- **656**: Multi-point navigation available, but some points invalid
- **657**: Only a single valid point; insufficient for multi-point navigation
- **658**: Multi-point navigation running
- **659**: Multi-point navigation not running
- **660**: Insufficient points for multi-point patrol

Version Upgrade

- **701**: No USB drive found
- **702**: Algorithm update package not found on USB drive
- **703**: Invalid update package on USB drive
- **704**: Update package corrupted
- **711**: No network connection
- **712**: Failed to download update package configuration file
- **713**: Failed to download update package
- **714**: Failed to calculate update package checksum
- **715**: Update package checksum invalid
- **716**: Update server offline
- **717**: Firmware package not found
- **762**: Failed to obtain network time
- **763**: Failed to set system time
- **788**: Failed to decompress update package

Infrared Recharge

- **901**: Recharge successful
- **902**: Infrared signal not detected during recharge
- **903**: Charging station not found
- **904**: Recharge timeout
- **905**: Recharge in progress
- **906**: Recharge cancelled
- **907**: Obstacle detected during backward recharge
- **908**: Waiting during recharge
- **910**: Navigation to charging station failed
- **912**: Obstacle at rear prevented docking with charging station
- **913**: Infrared signal not detected
- **914**: No infrared recharge signal detected
- **915**: Timeout while docking at charging station
- **916**: Approaching charging station
- **917**: Searching for charging station signal
- **918**: Recharge cancelled during navigation to charging station
- **919**: Recharge waiting
- **920**: Retry recharge attempt

Global Self-Localization

- **1001**: Self-localization in progress
- **1002**: Insufficient vertex information for area self-localization
- **1003**: Invalid specified pose input, not expressed as a single point
- **1004**: Self-localization unavailable outside of positioning mode
- **1005**: Insufficient or invalid radar data during localization
- **1099**: Unexpected self-localization failure, log review required

Active Notification Content and Field Content

system_monitor

Function: System sensor detection abnormal report

- **Topic:** Lidar
  - **Level:** 16
  - **Msg:** Error specific information
  - **Function Remark:** Lidar anomaly
- **Topic:** Chassis drive data
  - **Level:** 16
  - **Msg:** Error sensor name array
    - Sensor data corresponds to error codes, separated by semicolons
  - **Function Remark:**
    - Chassis sensor abnormal.
    - Each module is uploaded in the form of a concatenated string.
    - Mainly includes left motor, right motor, IMU, and battery.

safety_controller

Function: Safety protection related

- **Topic:** laser_match

    - **Level:** 4
    - **Msg:** The current match degree value is included in the data content
    - **Function Remark:** Current map matching threshold is low
    - **Level:** 1
    - **Msg:** Tip function switch
    - **Function Remark:** Map matching function switch prompt
- **Topic:** laser_data
    - **Level:** 4
    - **Msg:** Most radar data is invalid
    - **Function Remark:** Most of the current lidar data is invalid
    - **Level:** 1
    - **Msg:** Tip function switch
    - **Function Remark:** Laser data validity switch prompt

Related algorithm knowledge

System coordinate system

3D Cartesian Coordinate System (XYZ System)

- The **right-hand coordinate system** is used.
- Orientation:
    - Middle finger → **x-axis**
    - Index finger → **z-axis**
    - Thumb → **y-axis**
- The **x-axis** is defined as pointing forward.

Euler Angles

- Three Euler angles describe rotations around the three axes:
    - **Roll** → rotation around the x-axis
    - **Pitch** → rotation around the y-axis
    - **Yaw** → rotation around the z-axis
- Rotation is considered **positive in the counterclockwise direction**.

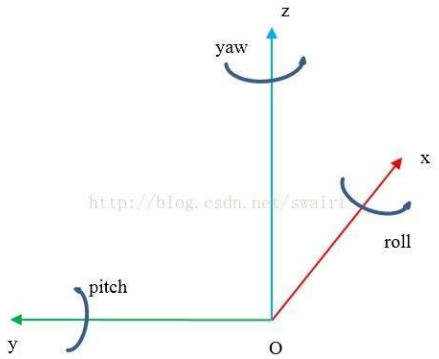


Figure 1 Right-hand coordinate system system

Define yaw (ψ), pitch (θ), and roll (φ) as the rotation angles around the Z axis, Y axis, and X axis, respectively

Transforming Radar Point Cloud to the Global Coordinate System

The radar point cloud data is originally expressed in the **robot coordinate system**, which means it cannot be directly displayed on the map. Therefore, a transformation step into the **global coordinate system** is required.

This extra step is necessary because keeping the point cloud relative to the **chassis position** makes it more useful for upper-level applications. Users can conveniently process the point cloud data with respect to the robot's current position. If the data were directly output in the global coordinate system, it would be harder to use the raw point cloud and even more troublesome to convert it back.

To transform the radar point cloud to the global coordinate system, the following information is required:

- **xRP, yRP**: The robot's coordinates in the global coordinate system.
- **heading**: The robot's current orientation (in radians).
- **x_laser, y_laser**: The x- and y-axis coordinates of the radar point cloud in the robot coordinate system.
- **xw, yw**: The transformed x- and y-coordinates of the radar point cloud in the global coordinate system.

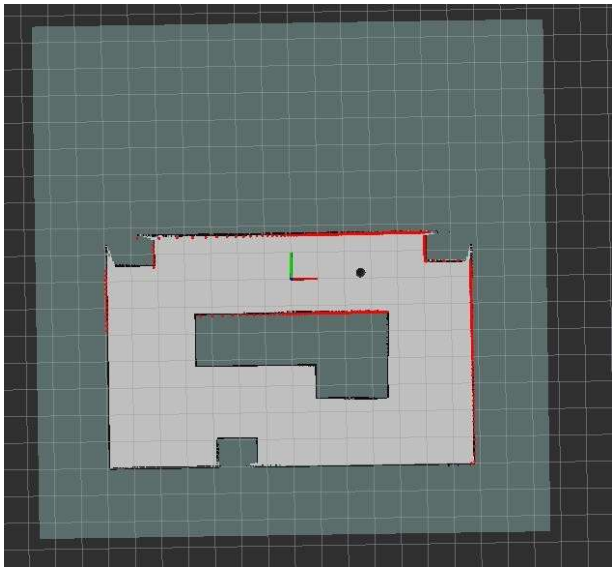
```
for (int i=0; i<x_laser.size(); i++)

{

    xw[i] = xRP + (x_laser[i] * cos(heading)) - (y_laser[i] * sin(heading));

    yw[i] = yRP + (x_laser[i] * sin(heading)) + (y_laser[i] * cos(heading));

}
```



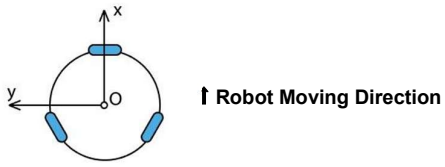
Red dot cloud data, and black markers are robot locations. If the ideal match is correct, the two combinations should be able to coincide with the actual map.

Differential Drive Speed Control

Robot Motion Control in the Right-Hand Coordinate System

The motion control of the robot body also follows the **right-hand coordinate system**. Since the robot uses **differential drive control**, its movement is defined by only two components:

- **Linear velocity (vx):** The speed of moving straight forward along the **x-axis**.
- **Angular velocity (wz):** The rotational speed (yaw) around the **z-axis**.



Euler Angle with a Quaternion Transformation

Rotation Representation in 3D Graphics

In 3D graphics, the most commonly used methods for representing rotation are quaternions and Euler angles. Compared with matrices, they offer advantages such as reduced storage requirements and easier interpolation.

This section mainly summarizes the conversion between these two representations, with calculation formulas based on the 3D Cartesian coordinate system.

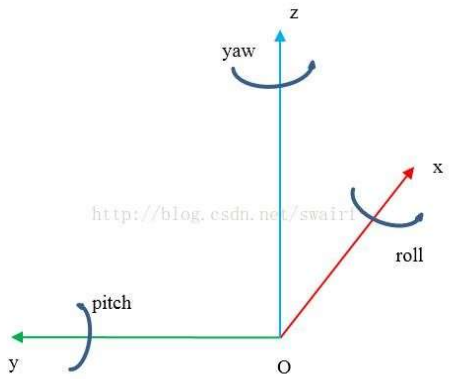


Figure 1 Right-hand coordinate system system

Define yaw (ψ), pitch (θ), and roll (φ) as the rotation angles around the Z axis, Y axis, and X axis, respectively

Euler Angle to Quaternionic Conversion

$$q = \begin{bmatrix} w \\ x \\ y \\ z \end{bmatrix} = \begin{bmatrix} \cos(\varphi/2)\cos(\theta/2)\cos(\psi/2) + \sin(\varphi/2)\sin(\theta/2)\sin(\psi/2) \\ \sin(\varphi/2)\cos(\theta/2)\cos(\psi/2) - \cos(\varphi/2)\sin(\theta/2)\sin(\psi/2) \\ \cos(\varphi/2)\sin(\theta/2)\cos(\psi/2) + \sin(\varphi/2)\cos(\theta/2)\sin(\psi/2) \\ \cos(\varphi/2)\cos(\theta/2)\sin(\psi/2) - \sin(\varphi/2)\sin(\theta/2)\cos(\psi/2) \end{bmatrix}$$

Conversion of the Quaternions to the Euler angle

$$\begin{bmatrix}\varphi\\ \theta\\ \psi\end{bmatrix}=\begin{bmatrix}\arctan\dfrac{2(wx+yz)}{1-2(x^2+y^2)}\\ \arcsin(2(wy-zx))\\ \arctan(\dfrac{2(wz+xy)}{1-2(y^2+z^2)})\end{bmatrix}$$

The result of Arctan and arcsin is $\left[-\frac{\pi}{2},\frac{\pi}{2}\right]$, which does not cover all orientations (the range

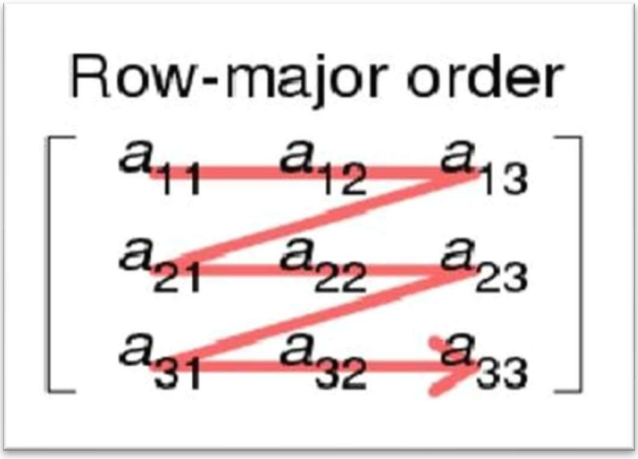
of values for θ angle $\left[-\frac{\pi}{2},\frac{\pi}{2}\right]$ is satisfied), so atan2 needs to be used instead of arctan.

$$\begin{bmatrix}\varphi\\ \theta\\ \psi\end{bmatrix}=\begin{bmatrix}atan2(2(wx+yz),1-2(x^2+y^2))\\ arcsin(2(wy-zx))\\ atan2(2(wz+xy),1-2(y^2+z^2))\end{bmatrix}$$

Transforming Map Pixel Coordinates into World Coordinates

Maps are stored in a row-major order mode for storage format.

As shown in the figures below.



Row-major order,
e.g., C (0-indexed)

Address	Access	Value
0	A[0][0]	a_{11}
1	A[0][1]	a_{12}
2	A[0][2]	a_{13}
3	A[1][0]	a_{21}
4	A[1][1]	a_{22}
5	A[1][2]	a_{23}

- When parsing image data, the real map data is stored in a list format.
- The data is read and parsed using **row-major order**. For example, the third element in the map data list represents the value at [0][2] / a13.
- **Pixel coordinates** are expressed as the row and column indices of the pixel in the image, which are integer values.
- **World coordinates** are expressed in a fixed coordinate system. Typically, the real-world position of the [0][0] pixel is given as a floating-point coordinate. The rest of the world coordinates are then calculated using the map resolution and the image width/height.
- **Note:** The display of the image may differ depending on the software used for rendering.
    - Usually, the first pixel of a map ([0][0]) is at the **top-left corner** of the image.
    - However, humans often interpret coordinates with the origin at the **bottom-left corner**.
    - In such cases, a **vertical flip (symmetry transformation)** may be required for consistency.

Pixel Coordinates to World Coordinates

Pixel coordinates: (px, py)
World coordinates: (wx, wy)
Origin: (origin_x, origin_y) = world coordinate of the first pixel
Resolution = map scale (e.g., 0.05 m per pixel)

Formula:
wx = origin_x + px × resolution
wy = origin_y + py × resolution

World Coordinates to Pixel Coordinates

World coordinates: (wx, wy)
Pixel coordinates: (px, py)
Origin: (origin_x, origin_y) = world coordinate of the first pixel
Resolution = map scale (e.g., 0.05 m per pixel)

Formula:
px = (wx − origin_x) ÷ resolution
py = (wy − origin_y) ÷ resolution